Laporan Pengerjaan: Rekonsiliasi Otomatis dengan n8n

Modul Implementasi AI untuk Keuangan, Topik 4: Rekonsiliasi Otomatis



Nama : Qois Nahwal Firdaus

NIM : 0110124014

Prodi : Sistem Informasi

Ai For Business

1. Latar belakang dan tujuan

Setiap perusahaan punya dua catatan yang seharusnya sama: catatan transaksi di sistem internal dan mutasi rekening dari bank. Kenyataannya dua catatan ini sering tidak kompak. Ada transaksi yang sudah dicatat tapi belum muncul di bank, ada juga yang sudah masuk bank tapi lupa dicatat. Mencocokkannya satu per satu secara manual itu melelahkan dan rawan salah lihat.

Di tugas ini kita membangun alur otomatis di n8n yang melakukan pekerjaan itu setiap malam. Alurnya mengambil data internal dan data bank, membersihkan formatnya, mencocokkan keduanya, lalu mengirim hasilnya ke Telegram supaya tim keuangan bisa langsung melihat mana yang beres dan mana yang perlu ditindaklanjuti.

Tujuan kita sederhana:

- Rekonsiliasi berjalan sendiri tanpa perlu diklik setiap hari.
- Selisih langsung ketahuan dan diberi tingkat prioritas.
- Laporan sampai ke Telegram dalam bentuk yang enak dibaca.

2. Gambaran besar alur kerja

Workflow kita terdiri dari 14 node. Secara garis besar alurnya seperti ini:

1. Jadwal jam 20.00 memicu dua jalur sekaligus.
2. Jalur pertama mengambil data internal dari Google Sheets, lalu disaring dan dibersihkan.
3. Jalur kedua mengunduh file CSV bank dari Google Drive, lalu dibaca dan dibersihkan.
4. Kedua jalur bertemu di node Merge.
5. Data gabungan dianalisis untuk menentukan status tiap transaksi.
6. Node If memisahkan yang cocok dan yang tidak cocok.
7. Masing-masing kelompok dirangkum jadi pesan, lalu dikirim ke Telegram.

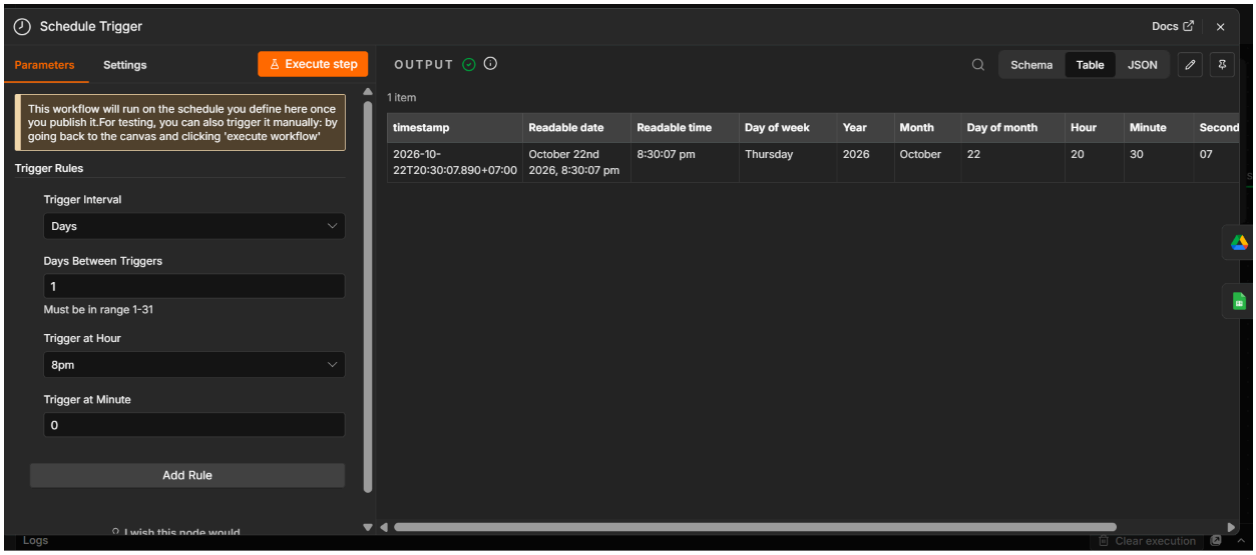
No	Node	Fungsinya
1	Schedule Trigger	Menjalankan workflow otomatis setiap hari jam 20.00
2	Get row(s) in sheet	Mengambil data transaksi internal dari Google Sheets (Sheet1)
3	Download file	Mengunduh file CSV mutasi bank dari Google Drive
4	Filter Baris	Menyaring dan membersihkan data internal
5	Extract from File	Mengubah isi file CSV menjadi baris-baris data
6	Clean Bank Data	Membersihkan data bank
7	Merge	Menggabungkan data internal dan data bank
8	Analisis Status	Menentukan status cocok atau tidak cocok untuk tiap transaksi

No	Node	Fungsinya
9	If	Memisahkan hasil MATCHED dan non-MATCHED
10	Rekap Data Match fix	Menyusun pesan rekap untuk transaksi yang cocok
11	Rekap Data UNMATCHED	Menyusun pesan rekap untuk transaksi yang tidak cocok
12	Wait	Memberi jeda sebelum pesan transaksi tidak cocok dikirim
13	Kirim Data Laporan yang cocok	Mengirim rekap transaksi cocok ke Telegram
14	Kirim Laporan Data yang tidak cocok	Mengirim rekap transaksi tidak cocok ke Telegram

3. Cerita pengerjaan, langkah demi langkah

Langkah 1: Menjadwalkan alur

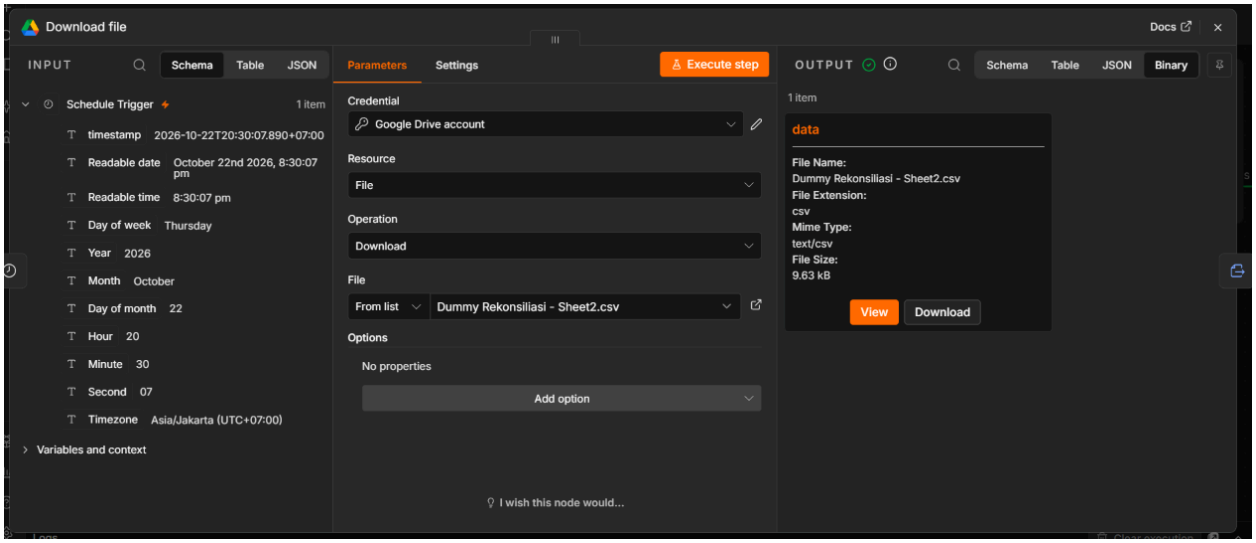
Kita mulai dari Schedule Trigger yang diatur jalan setiap hari pukul 20.00. Jam ini dipilih supaya transaksi hari itu sudah terkumpul sebelum direkonsiliasi. Satu trigger ini sekaligus menyalakan dua jalur paralel: satu untuk data internal, satu untuk data bank.

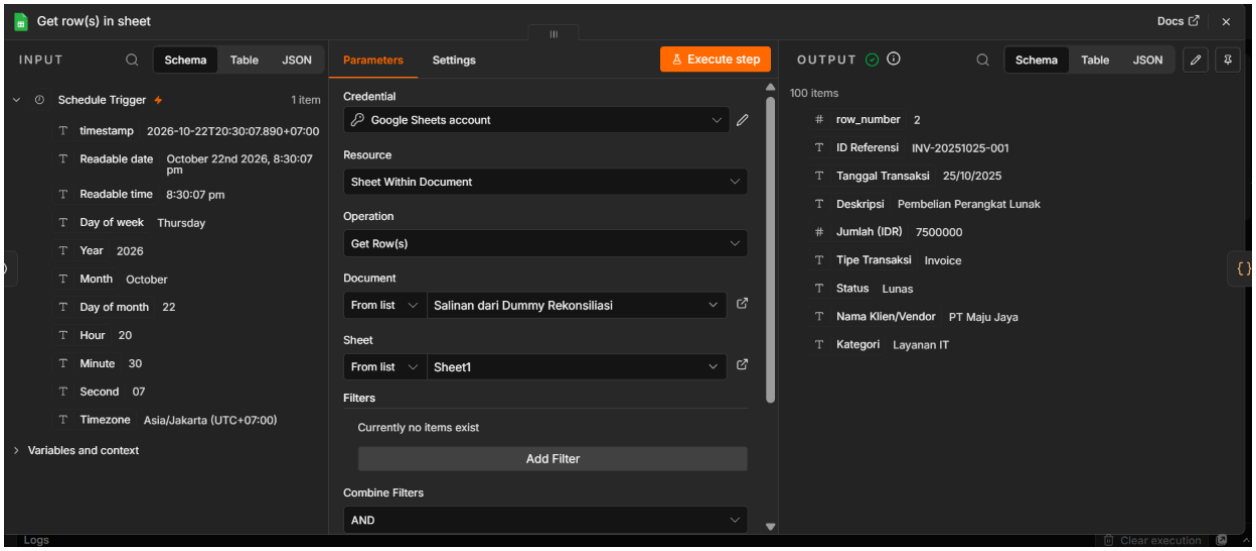


Langkah 2: Menyiapkan dua sumber data

Data internal kita ambil dari Google Sheets dengan node Get row(s) in sheet. Data bank kita ambil dari file CSV di Google Drive, lalu node Extract from File mengubahnya menjadi baris-baris yang bisa

diproses.

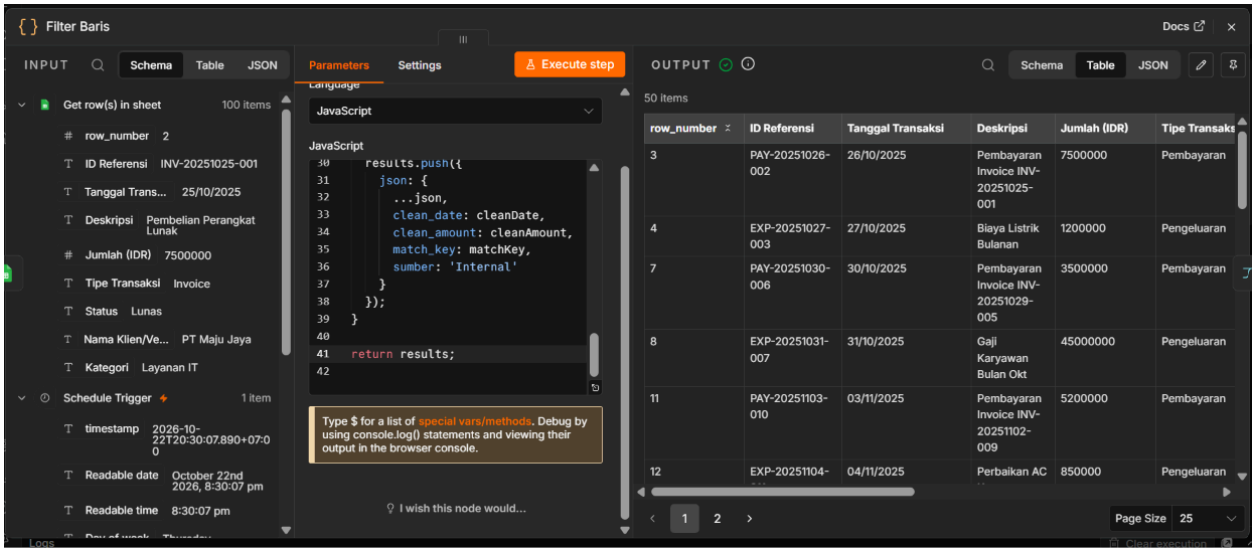




Langkah 3: Membersihkan data internal (node Filter Baris)

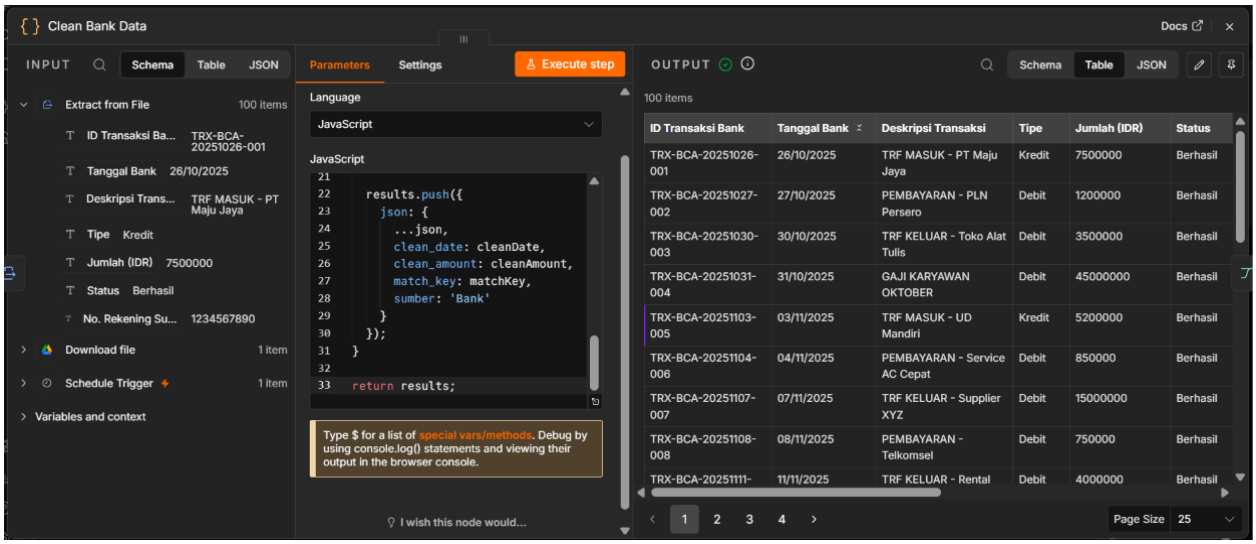
Data mentah jarang rapi, jadi di sini kita beresin dulu. Ada empat hal yang dilakukan:

- Menyaring tipe transaksi. Hanya baris bertipe Pembayaran, Pengeluaran, dan Penerimaan yang dipakai. Baris lain dibuang.
- Merapikan tanggal. Format DD/MM/YYYY kita ubah menjadi YYYY-MM-DD supaya seragam dengan data bank. Baris dengan tanggal yang tidak valid dilewati.
- Membersihkan angka. Titik dan koma pada nominal rupiah dihapus lalu diubah jadi angka bulat, supaya 1.500.000 dan 1500000 dianggap sama.
- Membuat match key, yaitu gabungan tanggal dan nominal, misalnya 2026-10-21_1500000. Kunci inilah yang nanti dipakai untuk mencocokkan. Setiap baris juga diberi label sumber: Internal.



Langkah 4: Membersihkan data bank (node Clean Bank Data)

Logikanya sama persis, hanya kolom tanggalnya memakai Tanggal Bank. Hasilnya juga punya match key dan diberi label sumber: Bank. Karena aturan pembersihannya kembar, kunci dari kedua sisi bisa dibandingkan secara adil.



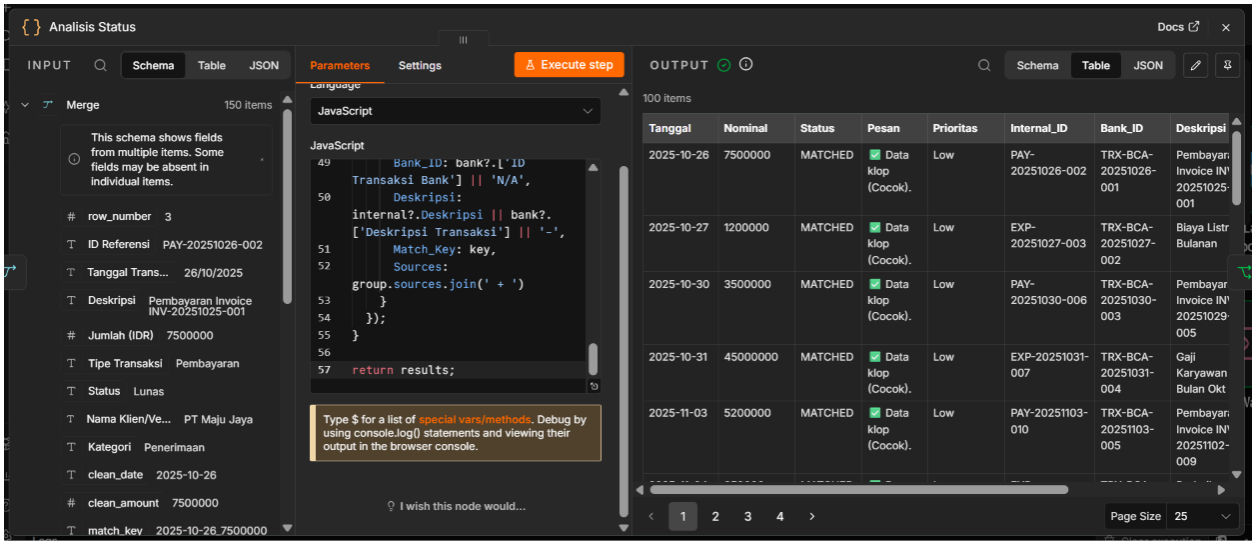
Langkah 5: Menggabungkan dan menganalisis

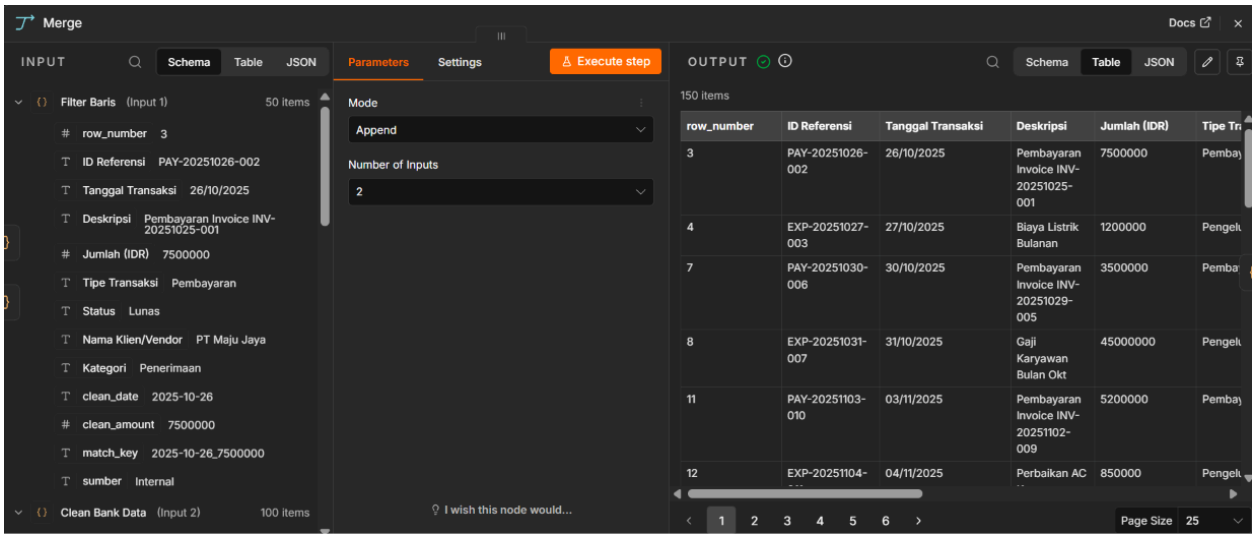
Node Merge menumpuk data internal dan data bank menjadi satu daftar. Setelah itu node Analisis Status mengelompokkan semua baris berdasarkan match key, lalu memeriksa dari mana saja asalnya.

Kondisi	Status	Prioritas	Artinya
Ada di Internal dan di Bank	MATCHED	Low	Data klop, tidak perlu tindakan
Hanya ada di Internal	MISSING IN BANK	High	Sudah dicatat, tapi tidak ada di bank. Perlu dicek, bisa jadi belum cair atau salah catat

Kondisi	Status	Prioritas	Artinya
Hanya ada di Bank	UNRECORDED INTERNAL	Medium	Ada di bank, tapi belum tercatat di internal

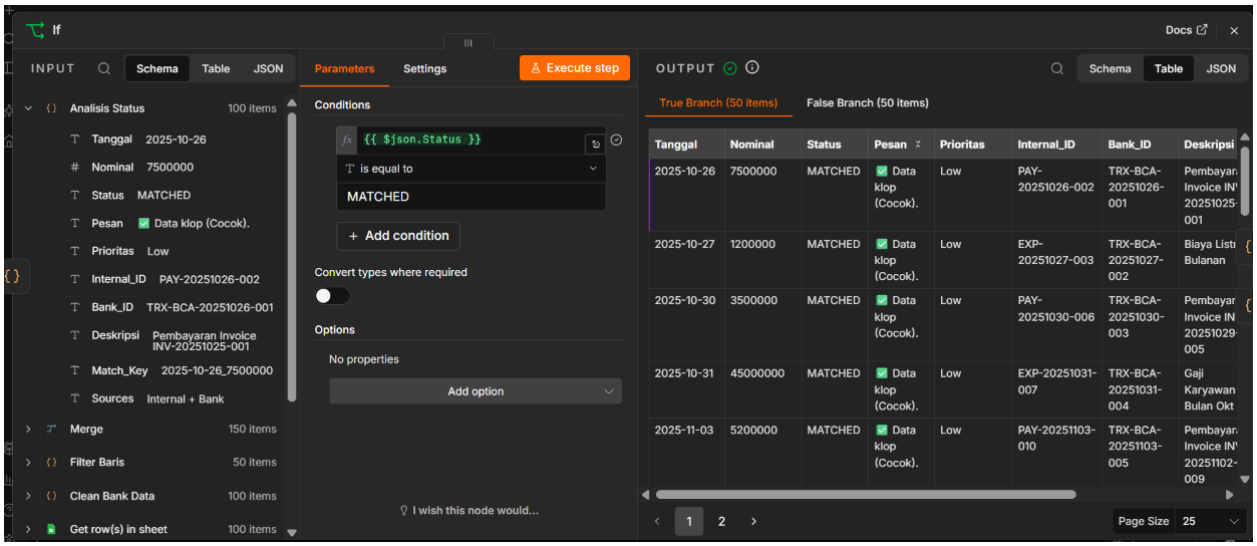
Untuk setiap kelompok, hasilnya berisi tanggal, nominal, status, pesan, prioritas, ID referensi internal, ID transaksi bank, dan deskripsi.





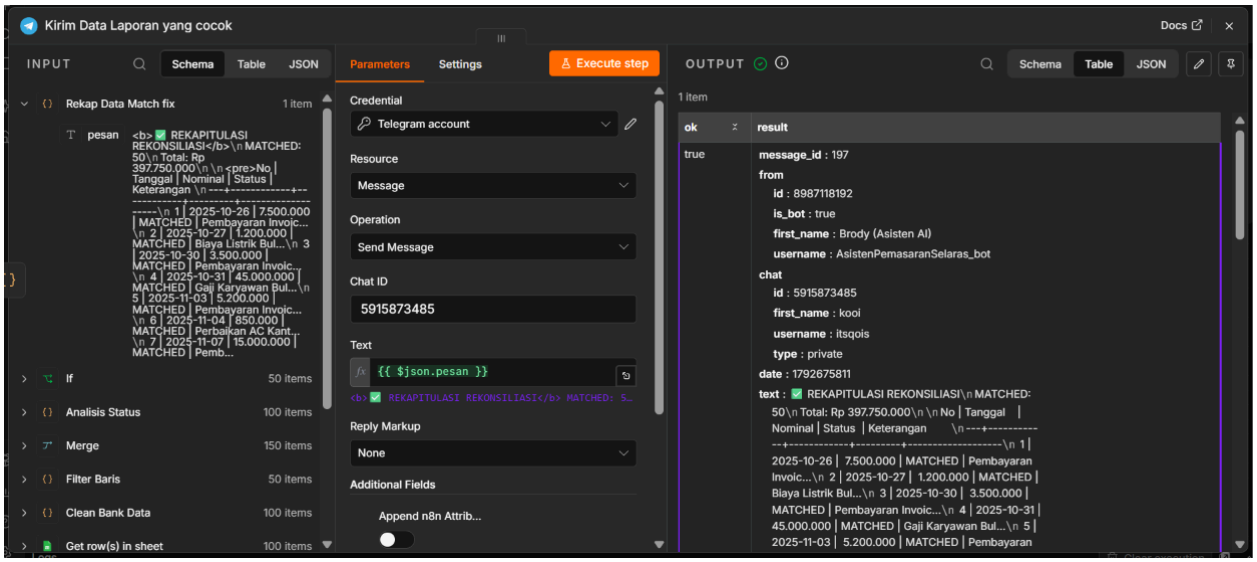
Langkah 6: Memisahkan hasil

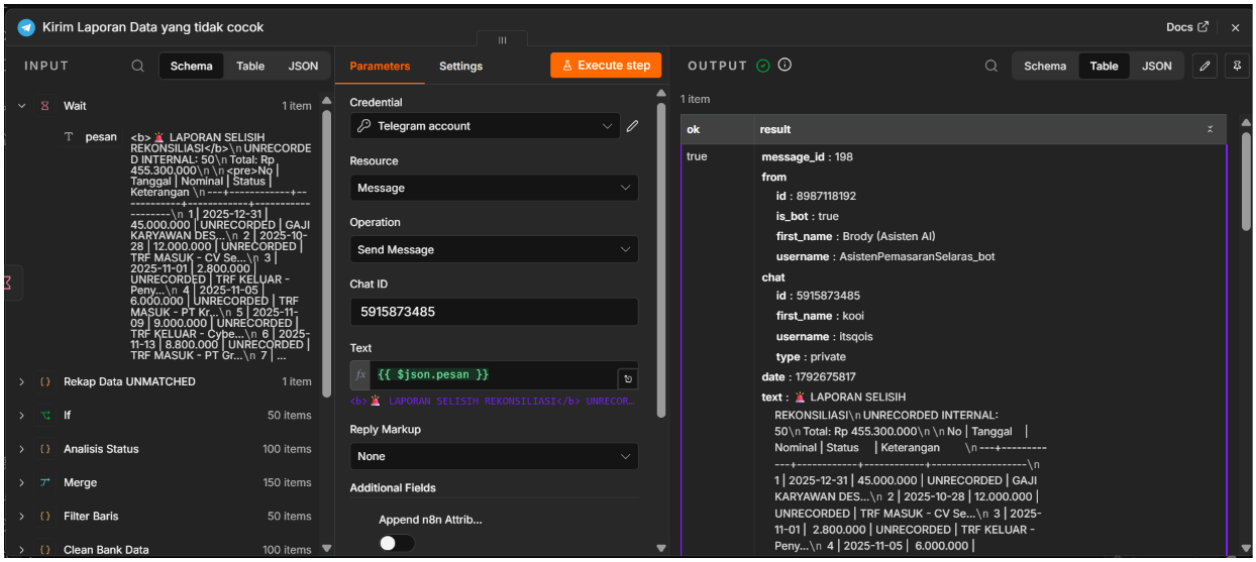
Node If memeriksa satu hal saja: apakah Status sama dengan MATCHED. Kalau ya, data masuk ke jalur rekap transaksi cocok. Kalau tidak, data masuk ke jalur rekap transaksi tidak cocok, yaitu MISSING IN BANK dan UNRECORDED INTERNAL.



Langkah 7: Menyusun dan mengirim laporan ke Telegram

Di tahap terakhir, dua node kode merangkum data menjadi teks dan dua node Telegram mengirimkannya ke chat yang sama. Bagian ini punya perbedaan penting dibanding panduan di modul PDF, dan kita bahas khusus di bagian berikut.





4. Perbedaan di node Telegram: satu pesan tabel, bukan 50 pesan

Yang dicontohkan di modul PDF

Berdasarkan modul, node Telegram dikirim langsung dari hasil analisis. Karena n8n memproses setiap baris sebagai satu item, node Telegram akan berjalan sekali untuk setiap item. Kalau datanya ada 50 baris, yang masuk ke Telegram adalah 50 pesan terpisah.

Yang kita buat

Kita menyisipkan satu node kode (Rekap Data Match fix dan Rekap Data UNMATCHED) di antara hasil analisis dan Telegram. Node ini mengumpulkan semua baris lalu menyusunnya menjadi tabel teks. Hasilnya satu pesan rapi, atau beberapa pesan lanjutan kalau tabelnya terlalu panjang.

Isi pesannya terdiri dari:

- Judul otomatis. Kalau ada selisih, judulnya laporan selisih rekonsiliasi. Kalau semuanya cocok, judulnya rekapitulasi rekonsiliasi.
- Ringkasan jumlah per status dan total nominal dalam rupiah.
- Tabel dengan kolom No, Tanggal, Nominal, Status, dan Keterangan. Teks dibungkus tag pre supaya tampil monospace dan kolomnya lurus.
- Pemecahan otomatis. Jika tabel melewati batas 3.500 karakter, tabel dipecah jadi pesan lanjutan bertanda 2/3, 3/3, dan seterusnya. Batas 3.500 dipilih supaya aman di bawah batas Telegram yang 4.096 karakter per pesan.

```
const items = $input.all().map(i => i.json);
if (items.length === 0) return [];

const esc = s => String(s ?? '-').replace(/&/g, '&amp;').replace(/</g, '&lt;').replace(/>/g, '&gt;');
const fmt = n => Number(n).toLocaleString('id-ID');
const cut = (s, n) => { s = String(s ?? '-'); return s.length > n ? s.slice(0, n - 1) + '…' : s; };
const short = { 'MATCHED': 'MATCHED', 'MISSING IN BANK': 'MISSING', 'UNRECORDED INTERNAL': 'UNRECORDED' };

// Judul otomatis sesuai isi data
```

```
const adaSelisih = items.some(r => r.Status !== 'MATCHED');
const judul = adaSelisih ? '🚨 LAPORAN SELISIH REKONSILIASI' : '✅ REKAPITULASI REKONSILIASI';

// Ringkasan
const count = {};
items.forEach(r => { count[r.Status] = (count[r.Status] || 0) + 1; });
const total = items.reduce((s, r) => s + (Number(r.Nominal) || 0), 0);
const ringkas = Object.entries(count).map(([k, v]) => k + ': ' + v).join(', ');
const head = '<b>' + judul + '</b>\n' + esc(ringkas) + '\nTotal: Rp ' + fmt(total) + '\n\n';

// Data tabel
const rows = items.map((r, i) => ({
  no: String(i + 1),
  tgl: r.Tanggal,
  nom: fmt(r.Nominal),
  st: short[r.Status] || r.Status,
  ket: cut(r.Deskripsi, 18)
}));
const cols = [['No', 'no', 'r'], ['Tanggal', 'tgl', 'l'], ['Nominal', 'nom', 'r'], ['Status', 'st', 'l'],
['Keterangan', 'ket', 'l']];
const w = cols.map(c => Math.max(c[0].length, ...rows.map(r => String(r[c[1]]).length)));
const pad = (s, width, a) => a === 'r' ? String(s).padStart(width) : String(s).padEnd(width);
const line = vals => vals.map((v, i) => pad(v, w[i], cols[i][2])).join(' | ');

const header = line(cols.map(c => c[0]));
const sep = w.map(x => '-'.repeat(x)).join('-+-');
const body = rows.map(r => line(cols.map(c => r[c[1]])));

// Pecah jika melebihi batas pesan Telegram
const LIMIT = 3500;
const base = head.length + header.length + sep.length + 20;
const msgs = [];
let cur = [];
let len = base;
for (const b of body) {
  if (len + b.length + 1 > LIMIT) { msgs.push(cur); cur = []; len = base; }
  cur.push(b);
  len += b.length + 1;
}
if (cur.length) msgs.push(cur);

return msgs.map((chunk, idx) => ({ json: {
  pesan: (idx === 0 ? head : '<b>Lanjutan (' + (idx + 1) + '/' + msgs.length + ')</b>\n\n')
       + '<pre>' + esc([header, sep, ...chunk].join('\n')) + '</pre>'
} }));
```

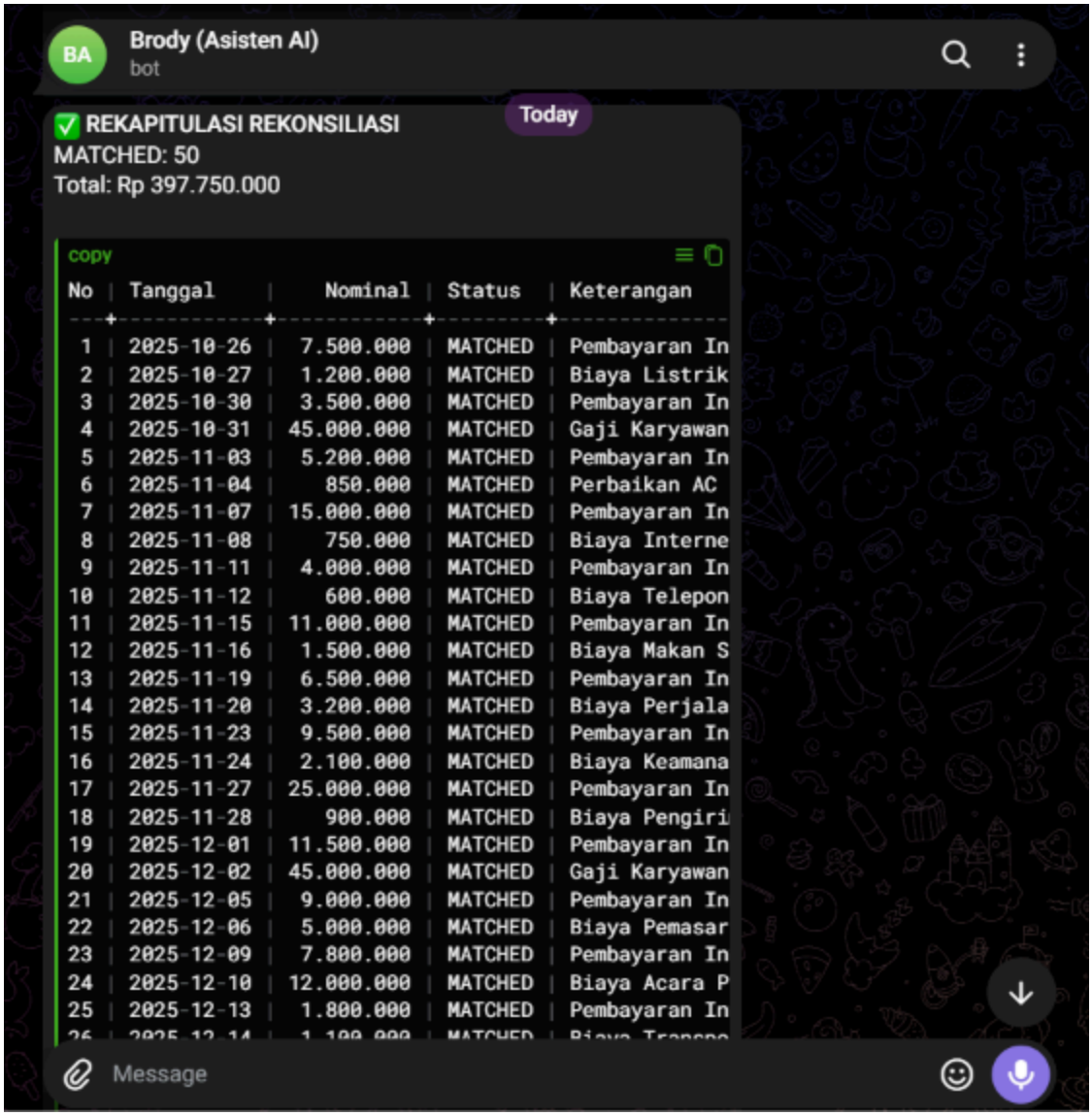
Perbandingan sekilas

Aspek	Versi modul PDF	Versi kita
Jumlah pesan untuk 50 baris	50 pesan, satu per baris	1 pesan tabel, atau beberapa kalau panjang
Node tambahan sebelum Telegram	Tidak ada	Node kode rekap untuk tiap jalur
Cara baca	Harus scroll dan membuka pesan satu per satu	Sekali lihat, semua baris sejajar dalam tabel
Notifikasi di HP	Berbunyi puluhan kali	Berbunyi satu atau beberapa kali saja
Ringkasan	Tidak ada	Jumlah per status dan total nominal di bagian atas

Aspek	Versi modul PDF	Versi kita
Risiko terbatasi laju kirim Telegram	Lebih tinggi karena banyak pesan beruntun	Jauh lebih rendah
Detail per baris	Bisa menampilkan semua kolom	Dipadatkan, keterangan dipotong 18 karakter

Kenapa kita pilih cara ini

- Enak dibaca. Tim keuangan butuh melihat gambaran utuh. Satu tabel jauh lebih cepat dipahami dibanding puluhan pesan yang bertumpuk.
- Tidak mengganggu. Satu notifikasi lebih manusiawi dibanding HP yang bergetar terus selama puluhan detik.
- Lebih aman dari batas Telegram. Mengirim banyak pesan beruntun ke chat yang sama bisa kena pembatasan laju. Dengan menggabungkan, risiko itu hampir hilang.
- Skalabel. Kalau besok datanya jadi 500 baris, kita tidak membanjiri chat dengan 500 pesan. Cukup beberapa pesan lanjutan.



Harga yang kita bayar

Setiap pilihan ada konsekuensinya, jadi ini juga perlu dicatat:

- Kolom keterangan dipotong supaya tabel muat di layar HP, jadi deskripsi panjang tidak tampil utuh.
- ID Referensi, ID Transaksi Bank, dan Prioritas tidak ikut ditampilkan di tabel. Detail lengkapnya tetap ada di data hasil analisis di n8n.
- Perlu dua node kode tambahan yang harus dirawat. Isinya mirip, jadi kalau mengubah format, keduanya perlu diubah.

Soal node Wait

Node Wait kita taruh di jalur transaksi tidak cocok, sebelum node Telegram. Fungsinya memberi jeda sebelum pesan itu dikirim, sehingga laporan transaksi cocok dan tidak cocok tidak terkirim bersamaan

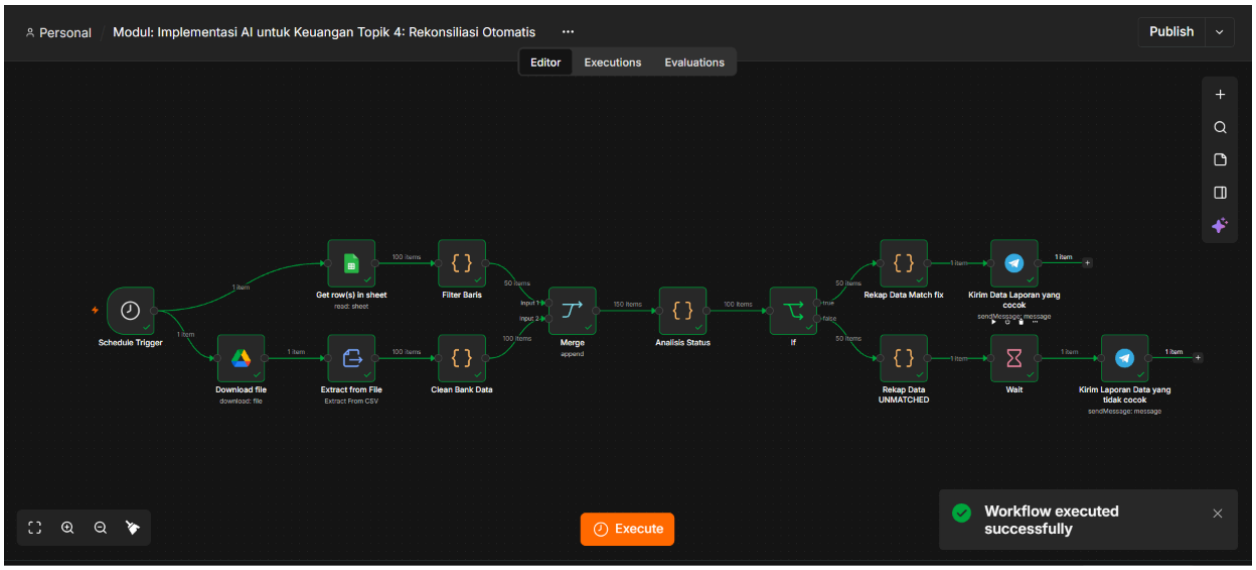
dan urutannya jelas. Setelah pesan digabung menjadi tabel, kebutuhan jeda ini memang berkurang, tetapi tidak merugikan, jadi kita biarkan saja.

5. Hal yang perlu diperhatikan

- Pencocokan memakai tanggal dan nominal saja. Dua transaksi berbeda dengan tanggal dan nominal yang sama akan dianggap satu kelompok. Untuk data yang lebih ramai, kita bisa menambahkan kolom lain, misalnya nomor referensi, sebagai pembeda.
- Pencocokan bersifat persis. Kalau tanggal di bank bergeser sehari karena proses kliring, transaksinya akan terbaca sebagai dua selisih: satu MISSING IN BANK dan satu UNRECORDED INTERNAL. Toleransi tanggal bisa jadi pengembangan berikutnya.
- Chat ID Telegram masih tertulis langsung di node. Kalau laporan perlu dikirim ke orang atau grup lain, nilainya harus diganti.
- Jadwal 20.00 mengikuti zona waktu yang diatur di n8n. Pastikan sesuai dengan WIB.

6. Kesimpulan

Kita berhasil membangun alur rekonsiliasi otomatis yang berjalan tiap malam, membersihkan dua sumber data dengan aturan yang seragam, mencocokkannya lewat match key, memberi status dan prioritas, lalu melaporkannya ke Telegram. Perubahan terbesar dari panduan modul ada di bagian Telegram: alih-alih mengirim 50 pesan satu per satu, kita merangkumnya menjadi satu tabel yang ringkas dan mudah dibaca. Hasilnya laporan terasa lebih seperti ringkasan kerja daripada tumpukan notifikasi.



Langkah pengembangan selanjutnya yang masuk akal: menambah toleransi selisih tanggal, memakai nomor referensi sebagai pembeda tambahan, dan menampilkan kolom Prioritas di tabel Telegram.

7.Lampiran

- Link Github :